

cnavier

Numerical Methodology and Scientific User Manual

cnavier project

Repository state: `main` at `5018e29bfff534b252a7ac9025e4ac43efd78ad`
6 October 2026

Abstract

cnavier is a compact two-dimensional incompressible Navier–Stokes solver written in C, with optional OpenMP and CUDA acceleration. The code advances the scalar vorticity equation and reconstructs a divergence-free velocity field from a stream function. Spatial derivatives are represented by finite-difference matrices of selectable interior order (second, fourth, or sixth), assembled into two-dimensional sparse operators through Kronecker products and applied in compressed sparse row (CSR) form. Time integration is explicit, using either forward Euler or classical fourth-order Runge–Kutta (RK4). The stream-function Poisson equation may be solved by Gauss–Seidel, successive over-relaxation (SOR), or a direct discrete-sine-transform solver implemented with FFTW3 on the CPU and cuFFT on the GPU.

This document describes the mathematical model, the exact discretizations present in the source code, solver sequencing, boundary treatment, stability considerations, verification and validation strategy, parallel backends, usage, and current numerical limitations. It is intended to be both a scientific description of the methodology and a practical reference for users and developers.

Contents

1	Introduction	4
1.1	Repository organization	4
2	Governing Equations	5
2.1	Nondimensional incompressible Navier–Stokes equations	5
2.2	Vorticity–streamfunction formulation	5
2.3	Why incompressibility is naturally enforced	5
3	Computational Grid and Data Layout	6
3.1	Domain and spacing	6
3.2	Grid convention	6
4	Finite-Difference Spatial Discretization	6
4.1	First derivative	6
4.2	Second derivative	7
4.3	Boundary closure and formal order	7
4.4	Two-dimensional operators by Kronecker product	8
5	Boundary Conditions and Wall Vorticity	8
5.1	Velocity boundary conditions	8
5.2	Vorticity boundary treatment	8
5.3	Streamfunction boundary condition	8

6	Vorticity Transport Discretization	9
7	Time Integration	9
7.1	Forward Euler	9
7.2	Classical fourth-order Runge–Kutta	9
8	Poisson Equation for the Streamfunction	10
8.1	Five-point discrete Poisson operator	10
8.2	Gauss–Seidel	10
8.3	Successive over-relaxation	10
8.4	Direct FFT/DST-I solver	11
9	Complete Timestep Algorithms	12
9.1	Euler sequence	12
9.2	RK4 sequence	12
9.3	Pseudocode	12
10	Stability and Timestep Selection	13
10.1	Implemented Courant check	13
10.2	Implemented viscous limit	13
10.3	Diffusive scaling	14
10.4	Centered advection	14
10.5	Recommended practice	14
11	Sparse Linear Algebra	15
12	Parallel Implementations	15
12.1	Backend structure	15
12.2	OpenMP	15
12.3	CUDA	16
12.4	GPU sine transform through odd extension	16
13	Diagnostics and Output	16
13.1	Discrete continuity diagnostic	16
13.2	VTK output	17
13.3	Centerline profiles	17
14	Verification and Validation	17
14.1	Analytic verification of the FFT Poisson solver	17
14.2	Residual verification of the iterative Poisson solvers	18
14.3	Short cavity integration tests	18
14.4	CPU–GPU equivalence tests	18
14.5	Spatial convergence against a manufactured solution	19
14.6	Physical validation against Ghia et al.	20
15	Performance Characteristics	20
16	Building and Running	21
16.1	Dependencies	21
16.2	Build modes	21
16.3	Tests	21
16.4	Runtime options	22
17	Default Numerical Configuration	22

18 Numerical Interpretation and Current Limitations	22
18.1 Mixed formal spatial order	22
18.2 Poisson operator differs from transport Laplacian	23
18.3 Boundary vorticity accuracy	23
18.4 Explicit stability	23
18.5 Steady-state convergence	23
18.6 Pressure is not solved	23
18.7 Geometry and dimensionality	23
19 Suggested Scientific Workflow	23
20 Possible Methodological Extensions	24
21 Final Remarks	24
A Mapping Between Equations and Source Code	25
B Reproducibility Record	25

1 Introduction

The objective of `cnavier` is to provide a small, inspectable computational fluid dynamics (CFD) code for two-dimensional incompressible viscous flow. Its reference problem is the square lid-driven cavity: three stationary no-slip walls and one tangentially moving wall. Despite the small code base, the project contains several numerical alternatives that are useful for study and benchmarking: multiple finite-difference stencil widths, two explicit time integrators, three elliptic solvers, a sparse-operator formulation, shared-memory parallelism, and an optional CUDA implementation of the complete time loop.

The code uses a *vorticity-streamfunction* formulation rather than the primitive-variable (u, v, p) equations. This choice removes pressure from the evolution system and replaces the pressure-velocity coupling problem by a scalar Poisson equation. In two dimensions, the streamfunction definition also gives a particularly strong discrete incompressibility property: when the discrete x - and y -derivative operators commute, the recovered velocity has zero discrete divergence to round-off.

The solver is designed primarily as an educational and experimental code rather than as a general-purpose production CFD package. The current implementation assumes a rectangular Cartesian domain, a Cartesian computational array, constant Reynolds number, explicit time stepping, and Dirichlet wall velocities. These constraints simplify the formulation sufficiently that each numerical operation can be followed directly from the governing equations to its C or CUDA implementation.

1.1 Repository organization

The main numerical components are:

- `src/main.c`: run configuration, command line, operator construction, time loop, diagnostics, and output;
- `src/backend.c`: one interface over the CPU and CUDA solvers, and the choice between them;
- `src/finitediff.c`: one-dimensional first- and second-derivative finite-difference matrices and CSR conversion;
- `src/linearalg.c`: dense storage utilities, CSR sparse matrices, sparse matrix-vector multiplication, and Kronecker products;
- `src/fluiddyn.c`: wall conditions, vorticity transport, Euler and RK4 integration, recovery of the velocity from the streamfunction, and full timesteps;
- `src/poisson.c`: Gauss-Seidel, SOR, and FFTW/DST-I Poisson solvers;
- `src/cudasolver.cu`: CUDA implementations of the time loop, sparse operators, elliptic solvers, and diagnostics;
- `src/threads.c`: the default number of OpenMP threads (one per physical core);
- `src/utils.c`: VTK output, centerline data for comparison with Ghia et al., and the available-memory check;
- `tests/test_solver.c`: CPU verification and CPU-GPU parity tests.

2 Governing Equations

2.1 Nondimensional incompressible Navier–Stokes equations

For an incompressible Newtonian fluid of constant density and viscosity, the nondimensional two-dimensional Navier–Stokes equations are

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \frac{1}{\text{Re}} \nabla^2 \mathbf{u}, \quad (1)$$

$$\nabla \cdot \mathbf{u} = 0, \quad (2)$$

where

$$\mathbf{u} = (u, v)^T$$

is the velocity, p is the kinematic pressure, and

$$\text{Re} = \frac{U_{\text{ref}} L_{\text{ref}}}{\nu}$$

is the Reynolds number.

For the default lid-driven cavity, $L_{\text{ref}} = 1$ and the lid speed is $U_{\text{ref}} = 1$, so the default nondimensional boundary data are

$$\begin{aligned} (u, v) &= (1, 0) && \text{on the moving lid,} \\ (u, v) &= (0, 0) && \text{on the other three walls.} \end{aligned}$$

2.2 Vorticity–streamfunction formulation

Define the scalar vorticity

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}. \quad (3)$$

Taking the curl of eq. (1) eliminates pressure and yields the vorticity transport equation

$$\frac{\partial \omega}{\partial t} + u \frac{\partial \omega}{\partial x} + v \frac{\partial \omega}{\partial y} = \frac{1}{\text{Re}} \left(\frac{\partial^2 \omega}{\partial x^2} + \frac{\partial^2 \omega}{\partial y^2} \right). \quad (4)$$

Introduce a streamfunction ψ by

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x}. \quad (5)$$

Substitution into eq. (3) gives

$$\nabla^2 \psi = -\omega. \quad (6)$$

Thus a timestep consists conceptually of transporting ω , solving the elliptic problem eq. (6), and differentiating ψ to obtain the velocity.

2.3 Why incompressibility is naturally enforced

At the continuous level, eq. (5) gives

$$\nabla \cdot \mathbf{u} = \frac{\partial^2 \psi}{\partial x \partial y} - \frac{\partial^2 \psi}{\partial y \partial x} = 0.$$

The code mirrors this property discretely. Let D_x and D_y denote the two-dimensional derivative matrices. The recovered velocity is

$$\mathbf{u}_h = \begin{bmatrix} D_y \psi \\ -D_x \psi \end{bmatrix}. \quad (7)$$

The discrete divergence is therefore

$$D_x D_y \psi - D_y D_x \psi. \quad (8)$$

Because the operators are built as Kronecker products acting in separate coordinate directions, they commute algebraically under the implemented layout. Consequently, the divergence diagnostic is expected to be near machine precision, apart from floating-point effects and details of the boundary rows. This behavior is explicitly tested by the project.

3 Computational Grid and Data Layout

3.1 Domain and spacing

The physical domain is specified by L_x and L_y and the computational array by $n_x \times n_y$. The implementation defines

$$\Delta x = \frac{L_x}{n_x - 1}, \quad \Delta y = \frac{L_y}{n_y - 1}, \quad (9)$$

n_x and n_y are independent; each may range from 8 through 16384.

Fields are stored as contiguous row-major arrays wrapped by the `mtrx` structure, with n_y rows of n_x values: element (i, j) lies at $y = i \Delta y$, $x = j \Delta x$ and has flat index $i n_x + j$, which is the ordering the Kronecker operators of eq. (17) assume. Earlier versions stored n_x rows of n_y values while the operators assumed the transpose; the two only agree when $n_x = n_y$, so non-square grids were rejected. Flattening a field is therefore a direct memory copy, and the two-dimensional derivative operations are implemented as sparse matrix–vector products.

3.2 Grid convention

The grid is nodal. Column j lies at $x_j = j \Delta x$, $j = 0, \dots, n_x - 1$, and row i at $y_i = i \Delta y$, $i = 0, \dots, n_y - 1$, so the first and last rows and columns of the arrays lie on the walls and n_x nodes span L_x with $n_x - 1$ intervals, as in eq. (9). All parts of the code use this convention:

1. wall velocities are imposed on the wall nodes, and the one-sided finite-difference closures are evaluated there;
2. all three Poisson solvers treat the $(n_x - 2) \times (n_y - 2)$ interior nodes as the unknowns and set $\psi = 0$ on the wall nodes (section 8.4);
3. centerline CSV output uses the node coordinates $i \Delta y$ and $j \Delta x$, and interpolates linearly onto $x = L_x/2$ (or $y = L_y/2$) when no node lies on the centerline, i.e. for an even number of nodes.

Earlier versions mixed conventions: the spacing was L/n , the FFT solver treated all $n_x \times n_y$ nodes as unknowns (placing $\psi = 0$ one cell outside the walls, while Gauss–Seidel and SOR placed it on them), and centerline coordinates were labelled $(i + \frac{1}{2})\Delta x$. Switching between solvers changed the flow, and the centerline profiles of the default case differed from Ghia et al. by up to 0.06; see section 14.6.

4 Finite-Difference Spatial Discretization

4.1 First derivative

The code supplies nominal second-, fourth-, and sixth-order centered stencils in the interior. For a uniform mesh and a sufficiently interior point,

Second order

$$f'_i \approx \frac{f_{i+1} - f_{i-1}}{2\Delta x} + \mathcal{O}(\Delta x^2). \quad (10)$$

Fourth order

$$f'_i \approx \frac{f_{i-2} - 8f_{i-1} + 8f_{i+1} - f_{i+2}}{12\Delta x} + \mathcal{O}(\Delta x^4). \quad (11)$$

Sixth order

$$f'_i \approx \frac{-f_{i-3} + 9f_{i-2} - 45f_{i-1} + 45f_{i+1} - 9f_{i+2} + f_{i+3}}{60\Delta x} + \mathcal{O}(\Delta x^6). \quad (12)$$

These coefficients correspond directly to the entries created by `SDiff1()`.

4.2 Second derivative

For the Laplacian terms in the vorticity transport equation, `SDiff2()` implements:

Second order

$$f''_i \approx \frac{f_{i-1} - 2f_i + f_{i+1}}{\Delta x^2} + \mathcal{O}(\Delta x^2). \quad (13)$$

Fourth order

$$f''_i \approx \frac{-f_{i-2} + 16f_{i-1} - 30f_i + 16f_{i+1} - f_{i+2}}{12\Delta x^2} + \mathcal{O}(\Delta x^4). \quad (14)$$

Sixth order

$$f''_i \approx \frac{2f_{i-3} - 27f_{i-2} + 270f_{i-1} - 490f_i + 270f_{i+1} - 27f_{i+2} + 2f_{i+3}}{180\Delta x^2} + \mathcal{O}(\Delta x^6). \quad (15)$$

4.3 Boundary closure and formal order

The requested `order` controls the widest interior stencil, but the boundary rows deliberately use narrower formulas. The closures are summarized in table 1.

Table 1: Effective stencil order near an array boundary in the current finite-difference matrices. The pattern is mirrored at the opposite boundary.

Requested order	Location	First derivative	Second derivative
2	boundary row	1st-order forward	2nd-order one-sided
2	interior	2nd-order centered	2nd-order centered
4	boundary row	1st-order forward	2nd-order one-sided
4	first interior row	2nd-order centered	2nd-order centered
4	deeper interior	4th-order centered	4th-order centered
6	boundary row	1st-order forward	2nd-order one-sided
6	first interior row	2nd-order centered	2nd-order centered
6	second interior row	4th-order centered	4th-order centered
6	deeper interior	6th-order centered	6th-order centered

Therefore, statements such as “sixth-order spatial discretization” should be read as *sixth-order centered interior stencils*, not as a demonstrated sixth-order global convergence rate. The global rate depends on the boundary closure, the Poisson discretization, the solution regularity, and the grid convention discussed above. Measured against a manufactured solution, it is second order for every derivative order (section 14.5).

4.4 Two-dimensional operators by Kronecker product

Let d_x, d_y denote the one-dimensional first-derivative matrices, d_{xx} and d_{yy} the one-dimensional second-derivative matrices, and I_x, I_y identity matrices. The code assembles

$$D_x = I_y \otimes d_x, \quad D_y = d_y \otimes I_x, \quad (16)$$

$$D_{xx} = I_y \otimes d_{xx}, \quad D_{yy} = d_{yy} \otimes I_x. \quad (17)$$

This is a convenient algebraic representation of tensor-product finite differences on a Cartesian grid.

The one-dimensional matrices are assembled directly in compressed sparse row (CSR) form, and so are the two-dimensional Kronecker products. For the widest sixth-order stencil, each derivative row has only a small fixed number of nonzero entries, so sparse matrix–vector multiplication scales linearly with the number of grid points rather than quadratically or cubically in the matrix dimension.

5 Boundary Conditions and Wall Vorticity

5.1 Velocity boundary conditions

The `wall\_bc` structure stores a constant (u, v) value for each of four array edges. At the beginning of every timestep, `apply\_wall\_bc()` writes those values directly into the edge rows and columns of the velocity arrays.

For the default cavity,

$$\mathbf{u}_{\text{lid}} = (1, 0), \quad \mathbf{u}_{\text{other walls}} = (0, 0). \quad (18)$$

No-slip and no-penetration are therefore represented through prescribed wall velocity.

5.2 Vorticity boundary treatment

A vorticity–streamfunction method requires a boundary value for ω that is consistent with the velocity boundary condition. The current code computes the velocity derivatives with the same finite-difference operators used elsewhere,

$$\omega|_{\partial\Omega} = (D_x v - D_y u)|_{\partial\Omega}, \quad (19)$$

and overwrites the boundary entries of ω before advancing the field, again at every RK4 stage, and once more at the end of the step. The update itself advances the boundary entries with the transport equation, which does not hold there, so the last overwrite replaces them with the wall vorticity of the new velocity; the field a step returns, and the one written to the VTK files, is consistent with its velocity at the walls too.

This differs from the frequently used Thom-type wall formula, which derives wall vorticity directly from a neighboring streamfunction value and wall velocity. In `cnavier`, the wall vorticity is instead obtained through one-sided/narrow derivative rows acting on the explicitly imposed velocity field. This design has the advantage of reusing one operator family for both interior and boundary derivatives, but it also means that the low-order boundary closure in table 1 directly influences the wall vorticity accuracy.

5.3 Streamfunction boundary condition

The Poisson solvers impose homogeneous Dirichlet conditions on the streamfunction. For a closed cavity, setting a constant value of ψ on every wall is consistent with no normal flow; the arbitrary constant is chosen to be zero:

$$\psi|_{\partial\Omega} = 0. \quad (20)$$

Tangential wall motion is then communicated to the interior through the vorticity boundary values rather than through a nonzero streamfunction value.

6 Vorticity Transport Discretization

Define the semi-discrete right-hand side

$$\mathcal{F}(\omega; u, v) = -u \odot (D_x \omega) - v \odot (D_y \omega) + \frac{1}{\text{Re}} (D_{xx} \omega + D_{yy} \omega), \quad (21)$$

where $\odot$ denotes pointwise multiplication. This is exactly the operation implemented in the CPU `dwdt()` routine and in the CUDA `rhs\_kernel`.

The convective terms are centered in the interior for all three selectable orders. Centered advection is nondissipative, which is attractive for accuracy but provides no built-in upwind damping. Consequently, timestep choice and adequate viscous resolution become important, particularly as Re increases.

7 Time Integration

7.1 Forward Euler

With `time\_scheme = 1`, the update is

$$\omega^{n+1} = \omega^n + \Delta t \mathcal{F}(\omega^n; u^n, v^n). \quad (22)$$

After the explicit vorticity update, the Poisson equation is solved once and the velocity is reconstructed from the new streamfunction. Thus Euler requires one elliptic solve per full timestep.

Forward Euler is first-order accurate in time. Its stability region does not include a finite segment of the imaginary axis, so centered advection by itself is not Euler-stable. Viscosity may stabilize a particular discretization and timestep, but the method should be treated as the low-cost/low-order option rather than the recommended high-Reynolds-number integrator.

7.2 Classical fourth-order Runge–Kutta

With `time\_scheme = 2`, the code uses classical RK4:

$$k_1 = \mathcal{F}(\omega^n), \quad (23)$$

$$k_2 = \mathcal{F}\left(\omega^n + \frac{\Delta t}{2} k_1\right), \quad (24)$$

$$k_3 = \mathcal{F}\left(\omega^n + \frac{\Delta t}{2} k_2\right), \quad (25)$$

$$k_4 = \mathcal{F}(\omega^n + \Delta t k_3), \quad (26)$$

$$\omega^{n+1} = \omega^n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \quad (27)$$

The velocity depends on the stage vorticity through eq. (6)–eq. (5). Accordingly, every call to the RK4 right-hand side performs a Poisson solve and recovers a stage-consistent (u, v) field. Four Poisson solves are therefore required for the four RK stages, followed by one final solve to make (u^{n+1}, v^{n+1}) consistent with ω^{n+1} . In total, RK4 performs five Poisson solves per timestep.

For a sufficiently smooth semi-discrete system, classical RK4 is fourth-order accurate in time. In the complete method that holds only if the boundary data move with the stages: the wall vorticity eq. (19) depends on the velocity, which changes from stage to stage, so it is recomputed

from each stage's velocity before that stage's derivatives are taken (section 5.2). Earlier versions set it once per timestep, from the velocity at the start of the step. The boundary data then lagged by up to one step, RK4 converged at first order only, and at the same step its error was about that of Euler. Measured on the lid-driven cavity (17×17 , $\text{Re} = 100$, ω on all nodes, walls included, at $t = 0.2$ against a run of the same scheme at $\Delta t/64$), the observed order is now 4.0 for RK4 and 1.0 for Euler, and at $\Delta t = T/64$ the error of RK4 is 1.2×10^{-8} against 2.5×10^{-2} for Euler. The test suite checks both orders. The steady state is essentially unaffected: at $t = 30$ the centerline profiles of the default case move by at most 1.2×10^{-4} , and their distance from the Ghia data changes from 0.0024/0.0074 to 0.0023/0.0073 (u/v).

8 Poisson Equation for the Streamfunction

At each velocity reconstruction, the code solves

$$\nabla^2 \psi = -\omega. \quad (28)$$

Three algorithms are available.

8.1 Five-point discrete Poisson operator

The iterative solvers and the spectral eigenvalues correspond to the conventional second-order five-point Laplacian. At an interior point, with i the row (y) and j the column (x) index as everywhere in this document,

$$\frac{\psi_{i,j+1} - 2\psi_{i,j} + \psi_{i,j-1}}{\Delta x^2} + \frac{\psi_{i+1,j} - 2\psi_{i,j} + \psi_{i-1,j}}{\Delta y^2} = f_{i,j}, \quad f = -\omega. \quad (29)$$

This is an important distinction: selecting fourth- or sixth-order finite-difference operators for vorticity transport and velocity differentiation does *not* change the order of the Poisson operator. The streamfunction solve remains based on the second-order discrete Laplacian.

8.2 Gauss–Seidel

Solving eq. (29) for the center value gives the Gauss–Seidel update

$$\psi_{i,j}^{\text{GS}} = \frac{\Delta y^2(\psi_{i,j+1} + \psi_{i,j-1}) + \Delta x^2(\psi_{i+1,j} + \psi_{i-1,j}) - \Delta x^2 \Delta y^2 f_{i,j}}{2(\Delta x^2 + \Delta y^2)}. \quad (30)$$

The serial CPU build sweeps lexicographically. The OpenMP build uses red–black ordering so that all nodes of one color can be updated in parallel. The CUDA implementation also uses red–black ordering.

8.3 Successive over-relaxation

SOR applies a relaxation factor β :

$$\psi_{i,j}^{(k+1)} = \beta \psi_{i,j}^{\text{GS}} + (1 - \beta) \psi_{i,j}^{(k)}. \quad (31)$$

The default program estimates an optimal relaxation parameter from the spectral radius of Jacobi iteration on the $(n_x - 2) \times (n_y - 2)$ interior nodes. With the weights of the five-point stencil (Δy^2 on the x neighbours, Δx^2 on the y neighbours), the slowest Jacobi mode is the lowest sine mode in each direction, so

$$\rho = \frac{\Delta y^2 \cos\left(\frac{\pi}{n_x - 1}\right) + \Delta x^2 \cos\left(\frac{\pi}{n_y - 1}\right)}{\Delta x^2 + \Delta y^2}, \quad (32)$$

$$\beta = \frac{2}{1 + \sqrt{1 - \rho^2}}. \quad (33)$$

which reduces to $\rho = \cos(\pi/(n-1))$ on a square grid with $\Delta x = \Delta y$. The five-point stencil is consistently ordered in both the lexicographic and the red-black sweep, so Young’s optimum β applies to the CPU and the GPU solvers alike.

The default maximum iteration count is 10000 and the shipped convergence threshold is 10^{-3} .

Stopping metric. The implementation function named `error()` computes

$$E^{(k)} = \sum_{i,j} \left| \psi_{i,j}^{(k+1)} - \psi_{i,j}^{(k)} \right|, \quad (34)$$

not a residual norm and not a root-sum-square norm. The console string currently labels this quantity “RSS error”, but scientifically it is an L_1 iterate-change metric. Verification tests separately compute the actual five-point Poisson residual.

Because the default tolerance can stop the iteration before full convergence, the sweep ordering can affect the returned approximation. This is why the serial lexicographic CPU result and the red-black GPU result need not be identical at the shipped tolerance; they approach the same solution as the tolerance is tightened.

8.4 Direct FFT/DST-I solver

The default `poisson\_type = 3` uses a two-dimensional discrete sine transform of type I (DST-I) of the $m_x \times m_y$ interior nodes, $m_x = n_x - 2$, $m_y = n_y - 2$. A DST-I of length m implies zeros at indices -1 and m , which are exactly the wall nodes, so the solver imposes $\psi = 0$ on the wall nodes like the iterative solvers and solves the same discrete problem. For each spectral mode (p, q) , the standard second-difference Laplacian has eigenvalue

$$\lambda_{p,q} = \frac{2 \cos\left(\frac{\pi p}{m_x+1}\right) - 2}{\Delta x^2} + \frac{2 \cos\left(\frac{\pi q}{m_y+1}\right) - 2}{\Delta y^2}, \quad (35)$$

with $p = 1, \dots, m_x$ and $q = 1, \dots, m_y$; note $m_x + 1 = n_x - 1$.

The algorithm is:

1. transform the right-hand side with a two-dimensional DST-I;
2. divide each transformed coefficient by $\lambda_{p,q}$;
3. apply the DST-I again;
4. multiply by the inverse transform normalization

$$\frac{1}{4(m_x + 1)(m_y + 1)} = \frac{1}{4(n_x - 1)(n_y - 1)}.$$

On the CPU the two-dimensional DST-I is applied as two passes of one-dimensional FFTW3 `RODFT00` transforms, first along the rows and then along the columns, in batches of eight rows or columns. The plans are created once at startup with `FFTW\_ESTIMATE` and reused. Two choices make the result deterministic. `FFTW\_ESTIMATE` picks plans without timing them; `FFTW\_MEASURE`, used previously, could pick a different plan on every run, and different plans round differently, so repeated runs at 513×513 differed in the last bits. And every row is transformed by the same plan whatever the number of threads, so serial and OpenMP builds give bit-identical results. The asymptotic cost is $\mathcal{O}(N \log N)$ with $N = n_x n_y$, compared with a potentially large number of $\mathcal{O}(N)$ sweeps for Gauss–Seidel or SOR.

Once Gauss–Seidel or SOR has converged, all three solvers return the same ψ ; the test suite checks agreement to 10^{-9} on a random right-hand side.

The FFT solver is “direct” with respect to the *discrete second-order Poisson problem*: once the transform is performed, no iterative convergence tolerance is required. It is not an exact solver for the continuous Poisson equation; the continuous truncation error remains that of the discrete Laplacian and the grid representation.

9 Complete Timestep Algorithms

9.1 Euler sequence

One CPU Euler timestep performs the following operations:

1. impose wall velocities on u and v ;
2. compute $D_y u$ and $D_x v$;
3. overwrite boundary vorticity using $\omega = D_x v - D_y u$;
4. compute $D_x \omega$, $D_y \omega$, $D_{xx} \omega$, and $D_{yy} \omega$;
5. update ω by forward Euler;
6. solve $\nabla^2 \psi = -\omega$;
7. recover $u = D_y \psi$ and $v = -D_x \psi$;
8. impose the wall velocities on copies of u and v and overwrite the boundary vorticity with $D_x v - D_y u$ of those copies, since the update advanced the boundary entries with the transport equation, which does not hold there. u and v themselves keep the values from ψ , whose divergence the continuity check measures.

9.2 RK4 sequence

One RK4 timestep begins with the same wall-velocity and wall-vorticity update. Each of the four stage right-hand-side evaluations then:

1. solves the Poisson equation for the stage streamfunction from the interior stage vorticity;
2. recovers a stage-consistent velocity field and imposes the wall velocities on it;
3. sets the wall vorticity of the stage from that velocity, eq. (19);
4. differentiates the stage vorticity;
5. evaluates the nonlinear convective and viscous terms.

After combining k_1 through k_4 , a fifth Poisson solve recovers velocity for the completed step, and the step ends like the Euler step: the wall velocities are imposed on copies of u and v , and the boundary vorticity is overwritten with $D_x v - D_y u$ of those copies.

9.3 Pseudocode

```
construct sparse DX, DY, DX2, DY2
allocate reusable solver workspace
initialize u, v, omega

for each timestep:
    impose wall velocities on u, v
    omega_wall = DX*v - DY*u on array edges
```

```

if Euler:
    rhs = -u*(DX*omega) - v*(DY*omega)
          + (1/Re)*(DX2*omega + DY2*omega)
    omega += dt*rhs
    solve Laplacian(psi) = -omega
    u = DY*psi
    v = -DX*psi

if RK4:
    for each RK stage:
        solve Laplacian(psi_stage) = -omega_stage    (interior)
        u_stage = DY*psi_stage
        v_stage = -DX*psi_stage
        impose wall velocities on u_stage, v_stage
        omega_stage_wall = DX*v_stage - DY*u_stage on array edges
        k_stage = RHS(omega_stage, u_stage, v_stage)
    omega = RK4 combination
    solve Laplacian(psi) = -omega
    u = DY*psi
    v = -DX*psi

(u_w, v_w) = (u, v) with wall velocities imposed
omega_wall = DX*v_w - DY*u_w on array edges

evaluate discrete divergence DX*u + DY*v
optionally write VTK

```

10 Stability and Timestep Selection

Because both time integrators are explicit, the timestep is restricted by the spectra of the discrete advection and diffusion operators.

10.1 Implemented Courant check

Before starting a run, `main.c` computes, for the default moving-wall velocity,

$$C_x = U_{\text{lid}} \frac{\Delta t}{\Delta x}, \quad C_y = U_{\text{lid}} \frac{\Delta t}{\Delta y}, \quad (36)$$

and aborts if either exceeds `max\_co = 1`, where U_{lid} is the largest wall speed. This is a useful sanity check but should not be interpreted as a complete stability proof. The actual interior velocity can differ from the lid velocity, and diffusion imposes a separate explicit restriction, which is checked as described next.

10.2 Implemented viscous limit

`max\_stable\_dt()` in `fluiddyn.c` bounds the timestep for the viscous term $\text{Re}^{-1}(D_{xx} + D_{yy})$. The interior second-derivative stencils are centered with coefficients of alternating sign, so the sum of their magnitudes is the value of the stencil symbol at the highest grid frequency, i.e. the largest eigenvalue magnitude:

$$\lambda_{\text{max}} = \frac{1}{\text{Re}} \left(\sum_k |a_k^x| + \sum_k |a_k^y| \right), \quad (37)$$

taken from the middle row of the one-dimensional operators (which is also the middle row of the two-dimensional ones), so the check runs before the two-dimensional operators are assembled. The run aborts unless

$$\Delta t \leq \frac{\rho}{\lambda_{\max}}, \quad \rho = \begin{cases} 2 & \text{forward Euler,} \\ 2.785293\dots & \text{classical RK4,} \end{cases} \quad (38)$$

ρ being the extent of each method's stability region along the negative real axis, and unless the Courant condition holds; the message names both limits and suggests the smaller one, rounded down so that the suggestion itself passes. For forward Euler the classical centered advection–diffusion condition $\Delta t \leq 2\nu/U^2 = 2/(\text{Re } U^2)$ is checked as well, but only as a warning, since it is conservative here (below); when an Euler run is refused, the suggested Δt is the smallest of all three limits, so that following it does not trigger the warning. Measured on the lid-driven cavity, the viscous bound is sharp: for orders 2, 4 and 6, both schemes and $n = 64$ and 128, runs at 0.95 times the bound stay bounded for 3000 steps and runs at 1.05 times diverge. The advection condition is conservative: at $\text{Re} = 1000$, $n = 64$, Euler stayed bounded up to about three times it. At the defaults ($n = 64$, $\text{Re} = 100$, sixth order) the bounds are $\Delta t \leq 0.00580$ for RK4 and $\Delta t \leq 0.00416$ for Euler.

10.3 Diffusive scaling

For the simple second-order Laplacian with forward Euler, the familiar two-dimensional diffusion restriction is approximately

$$\frac{1}{\text{Re}} \Delta t \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} \right) \lesssim \frac{1}{2}. \quad (39)$$

For $\Delta x = \Delta y = h$, this reduces to

$$\Delta t \lesssim \frac{\text{Re } h^2}{4}. \quad (40)$$

The precise limit changes with the higher-order second-derivative stencil and with RK4, but the important scaling remains

$$\Delta t = \mathcal{O}(h^2) \quad (41)$$

when explicit viscous stability controls the calculation. The repository benchmarks therefore use a timestep proportional to $1/n^2$.

10.4 Centered advection

The interior convective derivative is centered. Such a discretization contributes nearly imaginary eigenvalues for pure linear advection. RK4 has a finite stability interval along the imaginary axis and is therefore much better suited than Euler to centered advection. At high Reynolds number, however, neither time integrator substitutes for spatial stabilization if the grid is too coarse to resolve the vorticity gradients.

10.5 Recommended practice

For scientific runs:

- prefer RK4 over Euler unless Euler is being used deliberately for a comparison;
- reduce Δt under grid refinement, normally at least proportionally to h^2 until temporal error is demonstrated to be negligible;
- verify timestep independence by repeating selected calculations with $\Delta t/2$;
- monitor divergence, extrema, and visual smoothness of the vorticity field;

- treat a run that merely passes the implemented $C \leq 1$ check as *admissible input*, not automatically as a stable or converged simulation.

11 Sparse Linear Algebra

The dominant derivative operation is a CSR sparse matrix–vector multiplication

$$\mathbf{y} = \mathbf{A}\mathbf{x}. \quad (42)$$

For each row, `spmv()` loops only over the nonzero entries stored between consecutive elements of `row_ptr`. Since the finite-difference bandwidth is fixed, the cost is $\mathcal{O}(N)$ for $N = n_x n_y$.

The main dense field arrays are contiguous. Reusable flat buffers and RK4 work arrays are allocated once before the time loop; no per-stage dynamic memory allocation is needed. This improves both CPU cache behavior and GPU transfer strategy.

12 Parallel Implementations

12.1 Backend structure

A run is described by one `solver_config` (`fluiddyn.h`): grid size and spacing, Reynolds number, timestep, time scheme, Poisson solver settings, wall velocities and the four derivative operators. Both backends are initialised from it, and each keeps its own copy, taken at that moment: the CPU solver in its workspace (`rk4_ctx`, which also holds the work arrays and FFT plans) and the GPU solver (`gpu_init`), which uploads the operators and fields to the device. Changing the caller’s `solver_config` afterwards affects neither, and a test checks that both ignore such a change alike. `backend.c` chooses between them once, at startup, and exposes the same operations for both (one timestep, the divergence diagnostic, the host fields), so the time loop in `main.c` does not depend on where it runs. On the CPU the host fields are the solver’s own arrays; on the GPU they are copies filled from the device when asked for. A GPU run allocates no CPU solver workspace, and the memory check counts only what the chosen backend allocates.

12.2 OpenMP

Building with

```
make OPENMP=1
```

enables shared-memory parallel loops in the principal CSR operations, explicit right-hand-side loops, and suitable Poisson operations, including the batches of one-dimensional sine transforms. Loops over fewer than 2048 grid points (`OMP_MIN_WORK`) stay serial, since starting the threads would cost more than they save. Unless `OMP_NUM_THREADS` or a thread placement (`OMP_PROC_BIND`, `OMP_PLACES`) is set, the program uses one thread per physical core in its affinity mask instead of the OpenMP default of one per logical CPU: with two hardware threads per core the second adds nothing to these bandwidth-limited loops, and on an idle machine 16 threads on 8 cores were 25–40% slower than 8. Surplus threads are far more costly when other programs occupy the cores, since every parallel loop ends at a barrier where all threads wait for the slowest. Because the CSR products and transforms are limited by memory bandwidth, the speed-up levels off at about two from four threads on. Iterative elliptic solves use red–black coloring because a naive parallelization of lexicographic Gauss–Seidel would violate its data dependence.

Changing the sweep order is numerically relevant when the solver is stopped before convergence. Therefore, at a loose iterate-change tolerance, serial Gauss–Seidel/SOR and OpenMP red–black

Gauss–Seidel/SOR can return slightly different intermediate approximations even though they converge to the same discrete solution when the tolerance is sufficiently tight.

12.3 CUDA

Building with

```
make CUDA=1
```

adds a GPU backend. The four CSR derivative matrices and all primary fields are uploaded once and remain on the device throughout the time loop. CUDA kernels implement:

- wall velocity and wall-vorticity updates;
- CSR sparse derivatives;
- Euler and RK4 stage algebra;
- velocity recovery;
- red–black Gauss–Seidel/SOR;
- the FFT-based Poisson solver;
- the divergence diagnostic and reductions.

All solver arithmetic is double precision.

12.4 GPU sine transform through odd extension

cuFFT does not expose a native DST-I. The CUDA backend uses the identity that the DST-I of $x_1, \dots, x_m$ is $-\text{Im}$ of the real FFT of the odd extension $(0, x_1, \dots, x_m, 0, -x_m, \dots, -x_1)$ of length $2(m+1)$. The two-dimensional transform is done in two passes of batched one-dimensional transforms, as on the CPU: the odd extensions of the $n_y - 2$ interior rows (length $2(n_x - 1)$) are transformed together, the sine modes are read off and extended along the columns (length $2(n_y - 1)$), and those are transformed together. The code then divides by the same eigenvalues as eq. (35), applies the transform again, and scales by the DST normalization. An earlier version transformed one two-dimensional odd extension of size $2(n_y - 1) \times 2(n_x - 1)$, four times the grid; the batched passes are about 1.3 times faster from 129×129 upwards.

The red–black Gauss–Seidel and SOR solvers evaluate their stopping rule on the device: each sweep’s sum of $|\Delta\psi|$ is reduced and compared with the tolerance by a kernel, sweeps after the one that meets it do nothing, and the host reads the outcome once every 32 sweeps. The iteration count and the result are the same as with a check after every sweep.

This makes the GPU FFT algorithm mathematically equivalent to the CPU FFTW DST-I solver, up to floating-point round-off and the details of the FFT implementations.

13 Diagnostics and Output

13.1 Discrete continuity diagnostic

After every timestep, the CPU path computes

$$\epsilon \nabla \cdot u = D_x u + D_y v \tag{43}$$

and prints its maximum and minimum. The CUDA path computes the same field on-device and reduces it to its extrema. Since velocity is recovered from a streamfunction, these values are expected to remain close to floating-point round-off for the compatible operator pair.

This diagnostic should be interpreted as a consistency check on the streamfunction-to-velocity machinery. It is not, by itself, evidence that the momentum/vorticity equations are spatially converged.

13.2 VTK output

By default, vorticity is written to legacy ASCII VTK files in `output/`. The output interval is configurable and may be disabled with

```
./cnavier --output-interval 0
```

for performance measurements.

13.3 Centerline profiles

At the end of a run, the code writes four CSV files:

- `centerline\_u\_sim.csv`;
- `centerline\_u\_ghia.csv`;
- `centerline\_v\_sim.csv`;
- `centerline\_v\_ghia.csv`.

The reference data are the classical lid-driven-cavity values of Ghia, Ghia, and Shin at $Re = 100$ [1]. The project exports both numerical and reference profiles, but does not currently compute an automated interpolation-based error norm between them.

14 Verification and Validation

It is useful to separate *verification* from *validation*. Verification asks whether the implementation solves its stated discrete equations correctly. Validation asks whether those equations reproduce accepted physical/reference results to the required accuracy.

14.1 Analytic verification of the FFT Poisson solver

The test suite constructs a discrete sine eigenmode

$$f_{ij} = \sin\left(\frac{\pi p i}{n-1}\right) \sin\left(\frac{\pi q j}{n-1}\right) \quad (44)$$

on the nodes $i, j = 0, \dots, n-1$ (it vanishes on the wall nodes), with $n = 32$, $p = 3$, and $q = 5$. Since this is an exact eigenvector of the discrete Laplacian, the exact discrete solution is

$$\psi = \frac{f}{\lambda_{p,q}}. \quad (45)$$

The test requires the maximum relative difference to be no larger than 10^{-10} . This directly verifies the transform, eigenvalue division, and normalization.

14.2 Residual verification of the iterative Poisson solvers

For random right-hand-side data on a 24×24 grid, Gauss–Seidel and SOR are run with a tight iterate-change tolerance of 10^{-9} . The test then evaluates the actual five-point residual

$$r_{ij} = (L_h \psi)_{ij} - f_{ij} \quad (46)$$

and requires

$$\|r\|_\infty \leq 10^{-4}. \quad (47)$$

This is valuable because the production stopping criterion is based on successive iterate change rather than directly on r .

14.3 Short cavity integration tests

Both Euler and RK4 are exercised for 50 timesteps on a 32×32 lid-driven cavity at $\text{Re} = 100$, with sixth-order interior derivative operators, the FFT Poisson solver, and $\Delta t = 0.002$. The tests require:

- finite fields with bounded vorticity magnitude;
- a nontrivial velocity response to the moving lid;
- maximum discrete divergence no larger than 10^{-9} .

These are integration/smoke tests rather than accuracy tests against a fully converged reference solution.

14.4 CPU–GPU equivalence tests

When built with CUDA and run on a usable GPU, the test suite checks:

- host/device field round trips;
- continuity reductions on grids up to 300×300 ;
- all four CSR derivative operators on several grid sizes;
- FFT, SOR, and Gauss–Seidel Poisson solutions;
- Euler and RK4 full timesteps;
- multiple grid sizes, including 128×128 ;
- a case with four independently moving walls.

Typical comparison thresholds in the tests are 10^{-12} for CSR and FFT building blocks, 10^{-11} for full FFT-based timesteps, and 10^{-7} for converged iterative-solver comparisons. With OpenMP enabled, additional tests compare CPU and GPU red–black iterative solvers at the shipped production tolerance.

A CUDA test build with no usable GPU returns exit status 77 rather than reporting a false pass.

14.5 Spatial convergence against a manufactured solution

The cavity cannot measure the spatial order of the method: the lid is discontinuous at the top corners, the vorticity is singular there, and refinement on the cavity measures the singularity. The Ghia data are themselves a second-order solution on a 129×129 grid. The order is therefore measured against a manufactured solution,

$$\psi(x, y, t) = g(t) \sin^2\left(\frac{\pi x}{L_x}\right) \sin^2\left(\frac{\pi y}{L_y}\right), \quad g(t) = 0.2 + 0.1 \sin(2\pi t). \quad (48)$$

It has $\psi = 0$ and $u = v = 0$ on every wall, so it uses the solver's ordinary stationary no-slip walls and the $\psi = 0$ Poisson condition without any change. It is made an exact solution by an optional body force in the vorticity equation,

$$\frac{\partial \omega}{\partial t} = -u \frac{\partial \omega}{\partial x} - v \frac{\partial \omega}{\partial y} + \frac{1}{\text{Re}} \nabla^2 \omega + f, \quad f = \frac{\partial \omega_e}{\partial t} + u_e \frac{\partial \omega_e}{\partial x} + v_e \frac{\partial \omega_e}{\partial y} - \frac{1}{\text{Re}} \nabla^2 \omega_e, \quad (49)$$

where the subscript e denotes the exact fields from eq. (48), all in closed form. The force is the `forcing` callback of `solver_config`; it is unset in normal runs. The solver evaluates it at the time of each right-hand-side evaluation, so at t , $t + \Delta t/2$, $t + \Delta t/2$ and $t + \Delta t$ in RK4, and the CUDA backend fills it on the host and copies it to the device.

Each run starts from the exact state at $t = 0$, uses RK4 and the FFT solver at $\text{Re} = 100$, and is compared with the exact solution at $t = 0.25$. The time step is at most 2.5×10^{-3} and at most half the viscous stability limit; a run with a ten to thirty times smaller step gives the same errors to all digits shown, so the time error is negligible. The observed order between two grids is

$$p_{\text{obs}} = \log_2 \frac{\|e_h\|_\infty}{\|e_{h/2}\|_\infty}. \quad (50)$$

For derivative order 6 on the unit square:

Grid	ψ	u	ω , walls included	ω , interior
33^2	1.02×10^{-3}	2.97×10^{-3}	2.89×10^{-2}	2.01×10^{-2}
65^2	2.49×10^{-4} (2.03)	7.44×10^{-4} (2.00)	3.84×10^{-3} (2.91)	3.26×10^{-3} (2.62)
129^2	6.12×10^{-5} (2.02)	1.86×10^{-4} (2.00)	6.19×10^{-4} (2.63)	5.50×10^{-4} (2.57)
257^2	1.52×10^{-5} (2.01)	4.67×10^{-5} (2.00)	1.19×10^{-4} (2.37)	1.11×10^{-4} (2.31)
513^2	3.78×10^{-6} (2.01)	1.17×10^{-5} (2.00)	2.58×10^{-5} (2.21)	2.47×10^{-5} (2.16)

(maximum errors over the nodes, observed order in parentheses). The findings:

- The complete method is second order in space. ψ , u and v converge at 2.0 on every grid. ω converges faster on the coarse grids and approaches second order under refinement; its largest error is at the walls.
- Orders 4 and 6 give the same errors to three digits, so the sixth-order rows of the deep interior do not affect the result. The error is set by what all orders share: the five-point Poisson operator (eq. (29)) and the low-order rows next to the walls (table 1).
- Compared with order 2, orders 4 and 6 reduce the vorticity error by about $2.6\times$ at 513^2 (6.9×10^{-5} to 2.6×10^{-5} , walls included) and leave the velocity error essentially unchanged (1.02×10^{-5} for u with order 2, 1.17×10^{-5} with order 6).
- A 2×1 domain and a unit-square grid with $\Delta x = \Delta y/2$ show the same orders.

`make test` runs a short version: on 17^2 , 33^2 and 65^2 it requires the observed orders of ψ , u and v between 33^2 and 65^2 to be within 0.15 of 2 and that of ω to be at least 1.9, and it bounds the errors at 65^2 . It also checks that Euler and RK4 keep their first and fourth order in time with the time-dependent force, which fails if an RK4 stage evaluates the force at the wrong time. `make convergence` runs the full study up to 257^2 for orders 2, 4 and 6 and the two rectangular cases.

14.6 Physical validation against Ghia et al.

The code embeds the standard $\text{Re} = 100$ centerline velocity values from Ghia et al. [1] and writes them alongside the simulation profile. This supports a conventional cavity-flow validation workflow:

1. run to a demonstrably steady state;
2. interpolate the numerical centerline velocity to the Ghia sample coordinates, or vice versa;
3. compute L_2 , L_∞ , and selected pointwise errors;
4. repeat on successively refined grids;
5. repeat with smaller timesteps to separate spatial and temporal error.

For the default case ($\text{Re} = 100$, RK4, FFT solver, $t_f = 30$), the simulated centerline velocities interpolated to the 15 interior Ghia points give:

Grid	u on $x = 0.5$		v on $y = 0.5$	
	max error	rms error	max error	rms error
64×64 , mixed convention (before)	0.0594	0.0372	0.0595	0.0301
64×64 , nodal convention	0.0023	0.0014	0.0073	0.0039
65×65 , nodal convention	0.0026	0.0014	0.0076	0.0039
129×129 , nodal convention	0.0045	0.0024	0.0089	0.0049
257×257 , nodal convention	0.0050	0.0025	0.0091	0.0051

The solution has converged at these resolutions: at the Ghia points the 129×129 and 257×257 profiles differ by at most 4.5×10^{-4} (u) and 3.5×10^{-4} (v), an order of magnitude less than either differs from the reference. The remaining discrepancy is therefore not grid error of this code; it is of the order of the accuracy of the reference data themselves, which were computed on a 129×129 grid with a second-order scheme. The errors grow slightly from 64×64 to 129×129 : the smaller error of the coarse grid comes partly from its discretisation error cancelling against the difference to the reference, so 64×64 should not be read as the most accurate grid. The 257×257 run used $\Delta t = 3 \times 10^{-4}$.

`make regression` automates this comparison for the default case: the steady centerline profiles must be within 0.004 (u) and 0.010 (v) of the reference values, which are stored with the tests rather than taken from the program's output, and any value that is not a finite number fails. Grid-convergence rates are not measured on the cavity, for the reason given in section 14.5, but on a manufactured solution.

15 Performance Characteristics

The default FFT-based Poisson solver generally dominates the timestep cost, especially under RK4 where five elliptic solves are performed per step. Measured timings per timestep for the CPU, OpenMP and CUDA builds are kept in one place, the Performance section of `README.md`,

so that they are not maintained in two copies; they are implementation benchmarks on one machine rather than performance guarantees.

For transform performance, grid sizes for which $n-1$ factors into small primes are advantageous, because the sine transform of the $n-2$ interior nodes acts through FFT lengths of $2(n-1)$. Sizes such as 65, 129, 257, 513 and 1025 are therefore fast, and their neighbours can be several times slower: on the machine of the README benchmarks one serial FFT Poisson solve takes 6.7 ms at 255^2 ($2(n-1) = 508 = 4 \cdot 127$) against 1.6 ms at 257^2 (512), and 85 ms at 1023^2 against 31 ms at 1025^2 .

The GPU becomes increasingly advantageous as the grid grows because the fixed kernel-launch overhead is amortized over more work. Double-precision FFT throughput is a major cost on consumer GPUs, so larger speedups may be observed on hardware with stronger FP64 capability.

16 Building and Running

16.1 Dependencies

The CPU solver requires a C compiler, the math library, and FFTW3. OpenMP is optional. CUDA requires the NVIDIA CUDA toolkit and cuFFT.

On Ubuntu/Debian, a typical CPU setup is

```
sudo apt install build-essential libfftw3-dev
```

For CUDA development, the distribution toolkit or NVIDIA's toolkit packages may be used.

16.2 Build modes

```
make                # serial CPU
make OPENMP=1       # OpenMP CPU
make CUDA=1         # CUDA-enabled binary
make OPENMP=1 CUDA=1 # CUDA plus OpenMP CPU fallback/tests
```

The Makefile records the active compiler flags so that changing OpenMP/CUDA modes triggers the required rebuild automatically.

16.3 Tests

```
make test
make CUDA=1 test
make OPENMP=1 CUDA=1 test
make OPENMP=1 convergence # spatial convergence study, about 1.5 minutes
```

The test program returns 0 when all executed checks pass, 1 when any check fails, and 77 when a CUDA test build cannot execute its GPU checks because no usable device is present.

Further checks, each a `make` target run by continuous integration: `test-cli` (invalid command lines are refused, with the program's error status and message, without writing anything), `regression` (a short run against stored results and the steady default case against Ghia et al.), `test-asan` (AddressSanitizer and UndefinedBehaviorSanitizer), `valgrind` (no leaks, nothing still reachable), `format-check`, `cppcheck`, `tidy` (also over the CUDA and OpenMP branches of the C sources; the CUDA source itself is checked only by its compiler), and `WERROR=1` builds, which make `nvcc`'s warnings errors too. Continuous integration has no GPU, so the CPU-GPU comparisons run only where `make CUDA=1 test` is run on a GPU host.

16.4 Runtime options

The current binary accepts:

Option	Description
<code>-nx N</code>	Number of grid points in x .
<code>-ny N</code>	Number of grid points in y .
<code>-n N</code>	Same number of grid points in x and y .
<code>-dt DT</code>	Timestep.
<code>-tf TF</code>	Final simulation time.
<code>-output-interval N</code>	Write VTK every N steps; zero disables VTK output.
<code>-cpu</code>	In a CUDA build, force the CPU backend.
<code>-help</code>	Print command-line help.

Example:

```
mkdir -p output
./cnavier --n 127 --dt 6.2e-4 --tf 20 --output-interval 50
```

Parameters such as Reynolds number, finite-difference order, Poisson method, and time integrator are presently configured in `src/main.c` rather than through command-line flags.

17 Default Numerical Configuration

At the repository state documented here, the source defaults are summarized in table 3.

Table 3: Principal defaults in `src/main.c`.

Quantity	Default	Meaning
Re	100	Reynolds number
L_x, L_y	1, 1	domain lengths
n_x, n_y	64, 64	computational array size
Δt	0.005	timestep
t_f	30	final time
finite-difference order	6	sixth-order interior stencil
time_scheme	2	classical RK4
poisson_type	3	FFT/DST-I direct discrete solver
Poisson iterate tolerance	10^{-3}	GS/SOR iterate-change threshold
Poisson max iterations	10000	GS/SOR iteration cap
VTK output interval	20	steps between output files

18 Numerical Interpretation and Current Limitations

The following points are important when interpreting results scientifically.

18.1 Mixed formal spatial order

The transport derivatives may be sixth-order in the deep interior, but the boundary closures are lower order and the Poisson solve is based on a second-order discrete Laplacian. Measured against a manufactured solution, the complete algorithm is second order in space for every derivative order, and orders 4 and 6 give the same errors (section 14.5). Higher-order results would need a higher-order Poisson operator and wall closure together.

18.2 Poisson operator differs from transport Laplacian

The diffusion term in eq. (4) uses the selected $D_{xx} + D_{yy}$ operator, while the streamfunction equation always uses the standard five-point second-order Laplacian. This is a legitimate mixed discretization, but it should be described explicitly because the `order` setting does not uniformly change every spatial operator in the solver.

18.3 Boundary vorticity accuracy

Wall vorticity is reconstructed from differentiated wall velocities rather than from a dedicated high-order streamfunction wall formula. Its accuracy is therefore tied to the one-sided derivative closure and the array-edge interpretation.

18.4 Explicit stability

At startup a lid-speed Courant check and the viscous bound of the selected time integrator (computed from the assembled second-derivative operator, so it follows the grid, Reynolds number and finite-difference order) are enforced. These are stability bounds only; they say nothing about temporal accuracy, and users remain responsible for timestep-convergence studies.

18.5 Steady-state convergence

The default driver advances to a specified final time; it does not terminate based on a physical steady-state residual such as $\|\omega^{n+1} - \omega^n\|$ or a momentum residual. For steady cavity studies, t_f should be selected after checking that monitored fields and centerline profiles no longer change materially.

18.6 Pressure is not solved

Pressure is absent from the vorticity–streamfunction formulation. This is an advantage for the cavity velocity field but means that pressure output is not currently a native solved quantity. If pressure is required, it must be reconstructed from an additional pressure-Poisson or momentum-based relation with suitable boundary conditions.

18.7 Geometry and dimensionality

The current method is two-dimensional and Cartesian. Curvilinear grids, immersed boundaries, unstructured meshes, three-dimensional vorticity stretching, and general inflow/outflow boundaries are outside the present scope.

19 Suggested Scientific Workflow

For a reproducible numerical experiment, the following workflow is recommended:

1. Record the Git commit, compiler, optimization flags, FFTW/CUDA versions, CPU/GPU, and runtime backend.
2. Run `make test` for the chosen CPU build; for CUDA work, also run the GPU comparison suite on the actual GPU host.
3. Select a baseline grid and timestep conservatively.
4. Confirm temporal convergence by reducing Δt while holding the grid fixed.

5. Confirm spatial convergence with at least three grids while keeping temporal error smaller than spatial error.
6. For the lid-driven cavity, compare centerline profiles against Ghia et al. and report interpolation/error norms.
7. For steady solutions, verify that extending t_f does not materially change the solution.
8. Report the discrete divergence extrema as an implementation consistency diagnostic, but do not substitute them for solution-convergence metrics.

20 Possible Methodological Extensions

The current architecture makes several research-oriented extensions relatively natural:

- a consistent fourth- or sixth-order Poisson discretization, solved by an appropriate sparse/direct/iterative method;
- dedicated high-order wall-vorticity formulas;
- adaptive timestep selection from advective and diffusive spectral bounds;
- automated steady-state residual monitoring;
- automated Ghia interpolation and error norms in the test suite;
- grid-convergence-index (GCI) reporting;
- alternative convection schemes, such as upwind-biased, compact, WENO, or skew-symmetric energy-conserving formulations;
- multigrid for the Poisson equation when boundary conditions or geometries no longer permit a separable sine transform;
- pressure reconstruction as a post-processing step;

21 Final Remarks

`cnavier` implements a mathematically clean and computationally compact route to incompressible flow in two dimensions: evolve vorticity, invert a scalar Poisson equation, and differentiate a streamfunction to recover velocity. The code is particularly transparent because the spatial derivatives are explicit finite-difference matrices, the two-dimensional operators are tensor products, and both CPU and GPU backends expose the same algorithmic sequence.

The strongest parts of the present methodology are the exact discrete streamfunction enforcement of incompressibility, the direct DST-based Poisson option, reusable sparse operators, RK4 stage consistency, and unusually thorough CPU–GPU equivalence tests for a compact CFD code. The main scientific qualifications are equally clear: high-order accuracy applies primarily to interior transport derivatives; boundary closures are lower order; the streamfunction Poisson operator is second order; and quantitative benchmark/grid-convergence validation is not yet automated.

For educational CFD, numerical experimentation, backend comparisons, and as a foundation for further research-oriented development, these characteristics make the project both useful and unusually auditable. For publication-quality quantitative claims, the next step is not to complicate the code immediately, but to formalize its grid convention and add systematic temporal/spatial convergence and benchmark-error measurements.

A Mapping Between Equations and Source Code

Mathematical operation	Main implementation location
First-derivative stencils	SDiff1() in src/finitediff.c
Second-derivative stencils	SDiff2() in src/finitediff.c
Kronecker operators	skronecker() in src/linearalg.c; assembly in src/main.c
CSR derivative evaluation	spmv() in src/linearalg.c; spmv_kernel on CUDA
Wall velocity condition	apply_wall_bc() / wall_bc_kernel
Wall vorticity	step() / vorticity_bc_kernel
Vorticity RHS	dwdt(), euler() / rhs_kernel
RK4	rk4() / CUDA stage kernels in gpu_step()
Gauss–Seidel	poisson() / poisson_redblack()
SOR	poisson_SOR() / poisson_redblack()
FFT/DST-I Poisson	poisson_FFT() / poisson_fft()
Velocity recovery	velocity_from_vorticity() / velocity_kernel
Divergence diagnostic	main-loop CSR operations / continuity_kernel
Ghia centerlines	print_centerline() in src/utils.c
Verification suite	tests/test_solver.c

B Reproducibility Record

This document was prepared from the public repository

<https://github.com/leo-aa88/cnavier>

with `main` observed at commit

5018e29bff534b252a7ac9025ece4ac43efd78ad

on 6 October 2026. Numerical details in this manual describe the source code at that repository state; later commits may change defaults or algorithms.

References

- [1] U. Ghia, K. N. Ghia, and C. T. Shin, “High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method,” *Journal of Computational Physics*, vol. 48, no. 3, pp. 387–411, 1982.
- [2] J. H. Ferziger, M. Perić, and R. L. Street, *Computational Methods for Fluid Dynamics*, 4th ed., Springer, 2020.
- [3] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*, SIAM, 2007.
- [4] M. Frigo and S. G. Johnson, “The design and implementation of FFTW3,” *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005.